Jay Wanjare
wanjarejay25@gmail.com

Detailed Technical Approach

Objective

To develop a system that takes a simple text query as input and recommends the most relevant SHL assessments.

Data Collection and Web Scraping

A custom Python crawler was built using requests and BeautifulSoup.
The scraping process involved:

1. Navigating Listing Pages: Crawled paginated URLs on the SHL product catalog.
2. Link Extraction: Located the *"Individual Test Solutions"* table, skipped irrelevant ones, and extracted detail page links.
3. Adaptive IRT Capture: Retrieved the Adaptive IRT status from the listing table itself.
4. Detail Page Parsing: For each test link:
    - Extracted fields like title, description, languages, assessment length, test type, remote testing, and download link.
5. Crawling Strategy: Continued to the next page using an offset until no new links were found (3 consecutive empty pages).
6. Storage: All data was stored in a SQLite database and exported to CSV for further use.

Preprocessing and Feature Engineering

- Text Merging: Combined key textual fields (title, description, test type, etc.) into a combined_description.
- One-Hot Encoding: Applied to categorical fields like remote testing and test type for consistent representation.

Jay Wanjare
wanjarejay25@gmail.com

Embedding Generation with Google Gemini

- Embedding Creation: Each combined_description was passed through Google Gemini to generate a semantic embedding vector.
- Storage: These embeddings were stored alongside metadata for fast retrieval.
- Query Flow: A user query is also embedded using Gemini and compared to stored embeddings via cosine similarity.

Recommendation Engine

1. Query Processing: User text query is embedded.
2. Similarity Matching: Compared against all assessment embeddings using cosine similarity.
3. Ranking & Output: Top matching assessments are returned in ranked order.

Deployment (API Access)

- The system was deployed using FastAPI and hosted online at:
  https://assessment-recommendation-cadp.onrender.com/docs

  https://assessment-recommendation-sub.streamlit.app/

- The API accepts text input and returns a list of relevant assessments in JSON format.
- Supports real-time queries, and new assessments can be added via scraping + embedding.